before Turbo Stream does anything with that response, which is perfect for us.[44]

Here's code that does what we want. To start, if you look at the earlier code, I added a "data-animate-out": "animate__slideOutRight" to the turbo-frame tag. This makes the choice of animation data-driven and also allows us to distinguish between Turbo Stream removals where we want an animation, which will have this data element defined, and those where we don't want an animation, which won't.

Here, I'm adding an event listener for turbo:before-stream-render:

chapter_05/06/app/javascript/application.js

```
document.addEventListener("turbo:before-stream-render", (event) => {
  if (event.target.action === "remove") {
    const targetFrame = document.getElementById(event.target.target)
    if (targetFrame.dataset.animateOut) {
      event.preventDefault()
      const elementBeingAnimated = targetFrame
      elementBeingAnimated.classList.add(targetFrame.dataset.animateOut)
      elementBeingAnimated.addEventListener("animationend", () => {
        targetFrame.remove()
      })
    }
  }
})
```

(This code is based on sample code Nick Goodall posted to the Hotwire discussion board.)[45]

The event is passed to the callback function by Turbo; the event target is the code being returned. You get one of these callbacks for each Turbo Stream, so if your HTML response combines multiple requests, as ours does, you'll call this event multiple times.

The first thing we do is check the action of the stream event.target.action, which is the action= attribute of the incoming stream. We only care about removals for this part, so we only proceed if the action equals remove.

Next up we pull in the target frame ID, which is the target attribute of the event.target